

暗室藏书（DarkRoomLibrary）项目复盘

榜单、Agent、工程闭环与人的判断力

一、项目事实

维度	事实
项目名称	暗室藏书（DarkRoomLibrary）
技术栈	Spring Boot 2.7.18 + MyBatis-Plus 3.5.3.1 + JDK 17 + Vue 3 + Element Plus + ECharts + MySQL 8.0
项目规模	192 个 Java 文件 + 49 个前端文件 + 19 张表
测试体系	142 个后端测试 + 22 个前端单元用例 + 910 行 E2E 脚本 + 75 个以上 E2E 断言
项目文档	README + system-design.md + 功能模块图 + 人工验收清单
项目起点	一个基础 CRUD 脚手架 Demo
项目终点	业务、测试、文档、验收基本闭环的高完成度单体项目
开发者	计算机专业大二学生
开发时间	暑假约一周，每天约 2 小时，总计约 14 小时
初期工具	DeepSeek
最终主要工具	ChatGPT 5.5
审查环境	Trae Agent
参与评分模型	GLM 5.2 + DeepSeek V4 Pro
Agent 评分	9.3 / 10 (A)

需要说明的是，9.3 分不是项目作者的自我评价，而是由 Trae Agent 调用 GLM 5.2 与 DeepSeek V4 Pro 对项目进行审查后给出的评分。

这一评分能够说明：在该 Agent 的评价体系下，项目已经达到了较高的功能完整度、工程化程度和可交付水平。

但它仍然不能完全替代真实生产运行中的长期验证。因此，本文更准确地将项目定义为：

经双模型 Agent 审查获得 9.3/10 的高完成度、生产导向型全栈单体项目。

二、分析方法与证据边界

本文最初使用了“控制变量实验”这一说法，但严格来说，这并不是实验室条件下的随机对照实验。

它更接近：

同一用户、同一项目中的顺序性工具替换案例，以及围绕同一项目进行的多模型工程表现对照。

这里存在无法完全排除的变量：

- 用户在第一次失败后积累了经验；
- 项目需求在开发过程中逐渐清晰；
- 不同模型承担的具体阶段并不完全相同；
- Agent 的工具权限、上下文策略和推理预算可能不同；
- 模型版本、产品封装和停止策略也可能影响结果。

因此，本文不试图用一个项目证明所有模型在所有任务中的绝对排名。

本文能够证明的是：

1. 在暗室藏书这个具体项目中，不同模型的实际工程效果存在明显差异；
2. 这种差异并不完全符合公开榜单排序；
3. 单段代码生成能力与端到端工程能力不是一回事；
4. AI 的错误表达方式，会直接影响用户能否发现和纠正错误；
5. 当使用者缺乏判断力时，模型的工程缺陷会被进一步放大。

三、项目构建过程：从 DeepSeek 到 ChatGPT 5.5

3.1 最初使用 DeepSeek

项目最初尝试由 DeepSeek 在 CRUD 脚手架基础上继续构建。

DeepSeek能够生成：

- Vue 页面；
- Axios 请求；
- Spring Boot Controller；
- Service 与 Mapper；
- 数据库表；
- 登录注册模块；
- 权限与认证代码。

单独查看每个模块，代码通常具有一定的专业外观。

但项目很快出现了一个根本问题：

前端和后端分别写出来了，却无法真正连接。

前端调用的接口路径、请求参数、返回结构、认证头或数据字段，与后端实际提供的接口之间存在不一致。

最终，用户仍然需要亲自：

打开浏览器
→ 按 F12
→ 查看 Console 与 Network
→ 找到请求失败的位置
→ 判断是路径、参数、跨域、状态码还是数据结构错误
→ 再把错误交还给 AI 修复

这意味着 DeepSeek 实际完成的并不是全栈项目，而是：

生成前端代码，生成后端代码，然后将联调、测试和错误定位重新交还给用户。

3.2 “专家架构模式”的局部正确

DeepSeek在该项目中的典型行为，可以概括为一种“专家架构模式”：

前端专家负责生成前端
后端专家负责生成后端
数据库专家负责设计表
安全专家负责加入认证

每个局部都能够套用相应领域的专业模式。

但系统真正需要维护的是跨层契约：

跨层约束	可能出现的失配
接口路径	前端请求路径与 Controller 路径不一致
请求方法	前端使用 POST，后端定义为 PUT
参数名称	userId、id、user_id 混用
返回结构	前端期待数组，后端返回分页对象
认证请求头	Header 名称或 Bearer 前缀不一致
状态码语义	前端按 HTTP 状态判断，后端始终返回 200
分页参数	page/size 与 current/pageSize 不一致
时间格式	时间戳、ISO 时间和本地字符串混用
空值定义	null、空字符串与空数组含义不一致
异常结构	前端拦截器无法识别后端错误对象

全栈项目不是几个专业模块相加，而是所有模块共同服从同一套契约。

DeepSeek的问题不是完全不会写代码，而是：

它能够完成局部，却不能稳定维护整体。

也就是当时最直观的体验：

捡了芝麻，却丢了西瓜。

四、具体证据：只做一半的密码认证流程

项目开发过程中还留下了一条非常典型的故障记录：

“AI生成代码只会实现注册加密，忽略哈希比对时的编码、参数匹配细节，存密码看着完全正常，登录校验直接失效。AI偷工减料、只做一半流程。”

这不是抽象评价，而是一个具体、可观察的工程故障。

DeepSeek能够识别：

密码不能以明文形式写入数据库。

于是它在注册阶段加入了密码哈希。

数据库中的密码看起来也完全正常：

```
明文密码
→ 编码器处理
→ 哈希结果写入数据库
```

但它没有继续检查登录阶段是否采用了正确的比对逻辑。

对于 BCrypt 一类带随机盐的密码编码器，不能简单地重新编码用户输入，再与数据库哈希值直接比较：

```
passwordEncoder.encode(rawPassword).equals(storedPassword)
```

因为同一个明文密码经过多次编码，产生的结果通常不同。

正确方式应当使用编码器提供的匹配方法：

```
passwordEncoder.matches(rawPassword, storedPassword)
```

同时还必须保证：

- 明文和哈希值的参数位置正确；
- 注册与登录使用同一个编码规则；
- 数据库中没有发生额外字符编码或截断；
- 前端传递的字段与后端读取字段一致；
- 密码没有被重复编码；
- 登录流程确实执行了该匹配逻辑。

DeepSeek完成的是：

“注册代码里出现了密码加密。”

但系统真正需要完成的是：

注册
→ 哈希存储
→ 登录输入
→ 正确比对
→ 身份认证成功
→ 返回登录状态

这条链路少任何一环，认证功能都不成立。

因此，这个故障揭示的不是简单语法错误，而是典型的**半流程实现**：

模型把“生成了安全代码”误认为“安全功能已经完成”。

用户当时对此的概括非常直接：

胖鲸鱼贼懒，只做，不检查。

翻译成工程语言就是：

模型倾向于以代码生成作为任务终点，而不是以系统行为验证作为任务终点。

五、F12：代码生成器与工程 Agent 的分界

使用浏览器 F12 调试本身没有任何问题。

问题在于：究竟是谁在承担调试闭环。

5.1 DeepSeek的实际流程

生成前端代码
→ 生成后端代码

- 宣布功能基本完成
- 用户运行后发现无法连接
- 用户自己打开 F12
- 用户查看 Network 和 Console
- 用户判断错误类型
- 用户把错误反馈给模型
- 模型进行局部修补

在这一流程中，用户实际上承担了：

- 测试执行器；
- 错误采集器；
- 日志搬运工；
- 跨层定位器；
- Agent的外置反馈回路。

DeepSeek虽然以Agent形式工作，但其实际角色更接近：

带有文件写入能力的代码生成器。

5.2 ChatGPT 5.5的实际表现

项目切换到ChatGPT 5.5以后，前后端代码完成后没有持续出现同类型的基础联调问题。

这并不意味着ChatGPT永远不会产生Bug，而是说明在本项目中，它更稳定地维护了：

- 前后端接口路径；
- 请求方法；
- 请求参数；
- 返回字段；
- 认证协议；
- 页面读取结构；
- 异常处理方式；
- 数据库与实体映射；
- 页面、接口和权限之间的关系。

它更接近这样的工作方式：

- 生成代码
- 维护前后端契约
 - 检查上下文中的既有定义
 - 发现潜在冲突
 - 修正
 - 形成可以运行的闭环

因此，两者的差距并不是：

一个会写Vue，另一个不会写Vue。

而是：

DeepSeek更偏向模块级完成，ChatGPT 5.5更接近系统级完成。

可以用一句话概括：

DeepSeek把F12留给用户，ChatGPT把F12视为自己的工作。

六、为什么同样是Agent，实际效果仍然不同

“Agent”只是产品形态，不代表所有Agent拥有相同能力。

真实工程效果更接近：

Agent实际效果

=

基座模型能力

× 推理预算

× 上下文管理

× 工具权限

× 验证循环

× 停止条件

× 产品工程

可能造成差距的因素包括：

层面	具体影响
基座模型	能否同时追踪多个文件和跨层约束
推理预算	是否愿意在输出前进行更多检查
上下文管理	是否记得此前已经确定的接口和字段
Agent框架	是否会主动运行、读取错误并继续修复
工具权限	能否启动项目、执行测试、检查页面
停止策略	写完文件就停止，还是运行通过才停止
验证纪律	是否把测试通过作为完成标准
产品封装	是否会自动收集终端和浏览器反馈

因此，差距可能确实与模型“思维能力”和算力有关，但不能简单等同为：

思维差距就是算力差距。

更准确的判断是：

算力和推理能力决定模型能够追踪多少约束，Agent框架决定它是否会把这些能力用于持续验证，而停止条件决定它是在“代码写完”时结束，还是在“系统跑通”时结束。

七、GLM 5.2审查过程：从漏读到编造

项目完成以后，Trae Agent调用GLM 5.2和DeepSeek V4 Pro进行审查，并给出9.3/10的最终评分。

但在具体审查过程中，GLM 5.2出现了多处问题。

7.1 第一层：先判后查

初始判断	问题根因	实际事实
“前端测试只有7—10个用例”	只查看了部分单元测试目录	实际有22个单元用例，以及910行、75个以上断言的E2E脚本
“缺少认证有效性机制”	搜不到特定名称便下结论	项目存在 <code>/user/auth</code> 、页面挂载校验、路由守卫exp检查和401清理
“前端测试覆盖薄弱”	未完整查看E2E脚本	E2E覆盖组件、权限、HTTP集成、可访问性、响应式和视觉回归

需要特别区分的是：

项目确实没有标准的refresh token双令牌体系。

但这并不等于项目没有过期处理。

项目采用的是：

- 7天固定有效期；
- 页面加载时认证校验；
- 路由守卫检查；
- 401统一清理；
- 过期后重新登录。

因此正确结论应当是：

项目没有refresh token机制，但这是基于内部系统、单体部署和复杂度收益比作出的主动取舍，而不是遗漏了整个认证过期链路。

7.2 第二层：使用现代化Checklist替代场景判断

GLM建议	隐含标准	项目实际边界
Vue CLI应迁移Vite	新Vue项目默认Vite	项目来自Vue 2→3迁移，保留现有构建链降低风险

GLM建议	隐含标准	项目实际边界
应引入Pinia	Vue 3项目应有状态管理	项目刻意使用接口校验和sessionStorage，避免全局耦合
1101行组件必须拆分	SFC行数应保持较小	单人项目中拆分收益未必高于重构成本
JwtUtil静态字段是反模式	强调单测隔离	当前模式合法且已有142个测试验证
方法名前缀匹配应改注解	显式注解优于约定	注解收益有限，却增加样板和迁移成本
应增加sliding refresh	JWT系统应有双token	内部系统现有过期链路已满足项目目标

这些建议并非在所有场景中都错误。

例如：

- 1101行组件确实可能带来维护成本；
- 方法名匹配确实比注解更依赖命名纪律；
- Vite、Pinia和Spring Boot 3也都有各自优势。

问题在于：

GLM没有先计算这些问题在当前项目中是否已经造成真实损失，也没有评估迁移成本是否值得。

它识别到了一些理论上的设计代价，却直接将其升级为当前必须修复的缺陷。

这不是场景驱动审查，而是：

用统一现代化Checklist检查所有项目。

7.3 第三层：缺乏系统级成本意识

对于已经稳定运行的项目，任何技术升级都不应只看单项收益。

例如将Vue CLI迁移到Vite，需要评估：

- 构建配置；
- 环境变量命名；
- 代理配置；
- 静态资源路径；
- 打包目录；
- Webpack专用插件；
- 测试脚本；
- 部署流程；
- 团队认知成本。

它并不会必然要求Spring Boot 2.7同步升级到Spring Boot 3，但仍然可能引发一系列前端构建和部署链路调整。

真正的系统判断不是：

Vite更新，所以应该升级。

而是：

迁移能够解决什么已经存在的问题，其总收益是否大于风险和成本。

GLM的主要缺陷不是不知道新技术，而是：

把“技术上可以做”直接等同于“工程上应该做”。

7.4 第四层：在未核实前提下生成专业分析

最严重的问题，是GLM曾经基于一个不存在的前提，构造MySQL 5.7升级到8.0的连锁风险分析。

它使用了：

- CHECK约束；
- 字符集；
- 排序规则；
- `ONLY_FULL_GROUP_BY`；
- 驱动兼容；
- 数据迁移风险。

这些术语单独看都很专业。

但项目README已经明确写明：

MySQL 8.0。

项目从未存在MySQL 5.7升级到8.0的迁移路径。

因此，这一轮分析的问题不是知识点错误，而是：

前提没有核实

- 自动补全了一个不存在的项目历史
- 使用专业术语构造完整分析
- 以确定语气输出

这不是单纯的“没看见”。

它意味着：

模型在不知道事实时，没有停下来承认不知道，而是生成了一个看起来像知道的答案。

对于代码审查而言，这比漏掉一个问题更加危险。

八、第五层：封死用户的怀疑权

除事实误判外，GLM 5.2和DeepSeek在表达方式上还存在一个更隐蔽的问题：

它们经常用高度确定、不留余地的语气表达尚未核实的判断。

例如：

“项目缺少……”
“这里必须重构……”
“这是明显的反模式……”
“当前架构存在严重问题……”
“应当升级为……”

这些句子没有标记：

- 它究竟读了哪些文件；
- 哪些代码还没有检查；
- 结论是事实还是推测；
- 是否存在其他合理方案；
- 当前建议的收益和成本分别是什么；
- 用户是否应当先验证前提。

它把一个本应是：

“根据目前看到的部分代码，我怀疑……”

的问题，说成了：

“项目就是这样。”

这会制造一种**确定性错觉**。

8.1 ChatGPT 5.5的差异

在本项目协作中，ChatGPT 5.5并不是从不犯错。

它的关键差别是：

它更经常保留结论边界，不会轻易把尚未核实的判断说死。

它通常会区分：

- “已经确认”；
- “从当前代码看”；

- “可能存在”；
- “需要再检查”；
- “这是一种取舍，而不一定是缺陷”；
- “如果项目目标是……，那么……”；
- “这个建议有收益，但未必值得现在做”。

这种表达方式并不是软弱，也不是缺乏判断。

它实际上为用户保留了一项重要权利：

怀疑的权利。

当AI明确告诉用户某个结论仍有不确定性时，用户会继续：

- 查看代码；
- 运行项目；
- 检查日志；
- 对比文档；
- 要求模型证明；
- 拒绝不合理重构。

但当AI以绝对语气输出错误结论时，小白用户很可能直接相信。

8.2 对小白用户的实际后果

假设使用者缺乏工程经验，面对GLM或DeepSeek的确定性结论，他可能会：

- 相信项目没有测试，重新搭建一套测试体系；
- 相信必须迁移Vite，破坏原有构建链；
- 相信必须加入Pinia，引入不必要的状态复杂度；
- 相信必须升级Spring Boot 3，制造大规模迁移；
- 相信认证必须重写，破坏已有登录链路；
- 相信组件必须强拆，产生新的状态同步问题；
- 相信项目使用MySQL 5.7，并围绕不存在的问题修改配置；
- 在前后端无法连接时，误以为自己不会配置，而不是模型没有完成闭环。

最终，项目可能变成：

代码越来越多
依赖越来越新
架构越来越复杂
问题越来越分散
但系统仍然不能稳定运行

表面上，它比原项目“更现代”了。

实际上，它可能从9.3分项目退化为一个4—6分的复杂Demo。

因此，模型语气并不是无关紧要的交流风格。

它本身就是工程安全的一部分。

错误并不可怕；不允许用户怀疑的错误，才真正危险。

九、榜单反转：GLM 5.2为何在真实项目中不如DeepSeek V4 Pro

GLM 5.2在公开榜单上的表现非常强，甚至能够在部分评测中超过ChatGPT 5.5和DeepSeek V4 Pro。

但在暗室藏书的实际审查中，用户感受到的排序却是：

公开榜单：

GLM 5.2 > DeepSeek V4 Pro
部分项目中甚至 > ChatGPT 5.5

暗室藏书实际表现：

ChatGPT 5.5 >> DeepSeek V4 Pro > GLM 5.2

这不是普通分数误差，而是排序反转。

9.1 榜单通常测什么

许多软件工程榜单会给模型：

- 明确的问题描述；
- 指定仓库；
- 可运行测试；
- 明确的成功判据；
- 自动反馈；
- 有限范围内的修复目标。

模型需要完成的是：

问题已经定义
→ 找到相关代码
→ 生成修改
→ 让测试通过

而真实项目审查的第一步不是解决问题，而是：

确认问题到底存不存在。

它需要：

阅读项目定位

- 理解架构边界
- 检查代码与文档
- 恢复设计背景
- 区分缺陷、技术债和合理取舍
- 判断是否值得修改

这两类任务并不相同。

一个模型可能擅长：

在题目已经给出的情况下解决问题。

却不擅长：

判断题目本身是否成立。

9.2 榜单能力没有稳定迁移

暗室藏书案例说明：

榜单分数更高

≠

真实项目一定更可靠

更严格地说：

模型排名不具备跨任务的稳定传递性。

GLM 5.2可能在标准任务中拥有很强的：

- 补丁生成能力；
- 命令行操作能力；
- 单点问题解决能力；
- 测试驱动修复能力。

但在这次项目审查中，它没有稳定表现出：

- 事实核实纪律；
- 全仓库阅读能力；
- 场景化取舍能力；
- 系统级成本意识；
- 不确定性校准；
- 对自身结论的克制。

因此，这次案例不能证明GLM 5.2在所有任务上都弱于DeepSeek V4 Pro。

但足以证明：

它在榜单上的领先，没有稳定转化为暗室藏书项目中的真实工程领先。

十、三组对照分析

10.1 工具变化：DeepSeek与ChatGPT 5.5

相对稳定的条件

维度	情况
使用者	同一个大二学生
项目目标	同一个暗室藏书项目
技术栈	基本一致
初始基础	CRUD脚手架
主要变化	AI工具从DeepSeek切换为ChatGPT 5.5

实际结果

工具	项目表现
DeepSeek	前后端不能稳定连接，认证只完成半条链路，需要用户自行F12定位，最终放弃
ChatGPT 5.5	持续维护跨层契约，完成项目主体、测试、文档与验收闭环

可以得出的结论

在暗室藏书这一具体全栈项目中，ChatGPT 5.5比当时使用的DeepSeek更适合持续推进复杂工程任务。

差距不是单段代码写得是否漂亮，而是：

- 能否保持上下文；
- 能否维护跨层一致性；
- 能否主动检查；
- 能否完成端到端闭环；
- 能否让用户持续愿意使用。

这不是benchmark分数上的差距，而是：

用户是否会在真实项目中继续使用的差距。

10.2 使用者变化：同样的ChatGPT，不同的人

这里没有真正多人对照实验，因此不能把“多数人只能做出4—6分”写成已发生的实验结果。

但可以提出一个有充分工程依据的反事实判断：

同样使用ChatGPT，并不是所有人都能够获得同等质量的项目。

暗室藏书的最终质量，不只来自代码生成，还依赖使用者：

能力	项目证据
识别假完成	前后端代码都有，但连接不通，不承认任务完成
维护项目边界	明确定位为单机、小团队、增强型单体
拒绝沉没成本	发现DeepSeek路线存在根本问题后及时放弃
保持质量底线	建立142个后端测试、前端单测、E2E和验收清单
抵抗Checklist	不因“现代化”标签盲目迁移Vite、Pinia或Spring Boot 3
验证AI判断	不因模型语气确定就默认其结论正确
区分收益与成本	不为边际收益进行大规模重构
掌握否决权	AI建议可以参考，但不自动执行

因此，项目证明的不是“人比AI更会写代码”，而是：

当AI降低执行成本以后，项目瓶颈转移到了判断、验证、取舍和责任承担。

10.3 构建者与审查者的对照

对象	任务表现
学生 + ChatGPT 5.5	将CRUD脚手架演化为9.3分高完成度项目
GLM 5.2审查	首轮多处漏读、误判和虚构前提
DeepSeek V4 Pro审查	实际体验优于GLM，但仍存在确定性过强等问题

这不能简单证明：

人类构建者天然比AI审查者强。

因为构建者拥有完整历史上下文，而审查者面对的是已有仓库。

但它能够说明：

在缺少项目历史的情况下，审查者更应当降低确定性、主动核实事实，而不是用通用模式填补自己不知道的部分。

GLM恰恰做反了：

掌握的上下文更少
→ 表达却更加确定

这构成了审查方法上的根本缺陷。

十一、项目成功的三层结构

暗室藏书的成功可以概括为：

项目质量
=
工具能力
× 人的判断
× 迭代闭环

第一层：工具能力

ChatGPT 5.5降低了：

- 多文件代码生成成本；
- 前后端联调成本；
- 测试补充成本；
- 文档整理成本；
- 重构与排错成本。

缺少足够好的工具，项目可能在早期前后端失配阶段停滞。

第二层：人的判断

人决定：

- 项目真正要解决什么；
- 哪些功能已经完成；
- 哪些只是表面存在；
- 哪些复杂度值得引入；
- 哪些建议应当拒绝；
- 哪个工具已经不值得继续使用；
- 哪些AI结论需要再次核实。

缺少判断力，即使模型生成大量代码，也可能只是一个不能稳定运行的复杂Demo。

第三层：迭代闭环

项目不是一次生成完成的，而是：

```
生成
→ 运行
→ 发现问题
→ 检查
→ 修正
→ 测试
→ 再运行
→ 文档化
→ 人工验收
```

缺少这一层，项目会停在“看起来已经写完”的阶段。

十二、关于“正确的克制”

暗室藏书中最有价值的工程能力之一，是没有为“看起来先进”盲目增加复杂度。

例如：

- 不因为Vue 3就必须迁移Vite；
- 不因为存在用户状态就必须加入Pinia；
- 不因为Spring Boot 3更新就立即迁移；
- 不因为JWT就必须设计refresh token体系；
- 不因为组件超过某个行数就机械拆分；
- 不因为注解更显式就必须替换简单约定；
- 不因为AI建议重构就默认重构一定有收益。

但“什么都不做”并不自动等于克制。

真正的工程克制必须建立在四个问题上：

1. 这项技术解决什么问题？
2. 当前项目是否真的存在这个问题？
3. 引入它需要付出什么成本？
4. 不引入它的风险是否可以接受？

因此，最准确的说法不是：

不升级就是正确。

而是：

知道什么时候值得升级，什么时候不值得。

真正稀缺的不是拒绝复杂度，而是：

判断某项复杂度在什么时候值得引入。

十三、关于AI语气与工程安全

AI工程能力不仅体现在代码中，也体现在它如何表达自己的判断。

一个可靠的工程助手应当区分：

表达层级	示例
已确认事实	“README明确使用MySQL 8.0。”
高概率判断	“从当前调用链看，问题很可能出在认证头不一致。”
待核实假设	“我尚未看到完整E2E脚本，需要继续检查后才能评价覆盖率。”
工程取舍	“拆分组件可能提升维护性，但在单人项目中未必值得立即处理。”
建议	“可以考虑迁移Vite，但目前没有证据表明这是必要改动。”

当模型把所有层级都写成确定事实时，就会抹掉用户的判断空间。

这对小白尤其危险。

一个缺乏经验的用户很难区分：

- 专业术语；
- 真实事实；
- 通用建议；
- 模型猜测；
- 虚构前提。

如果模型又以不允许反驳的语气输出，用户就可能把错误结论直接转化为代码修改。

因此：

保留用户的怀疑权，不是语言风格问题，而是AI工程安全的必要条件。

ChatGPT 5.5在本项目中的一个重要优势，正是它更经常允许用户继续怀疑、追问和验证。

它不是永远正确。

但它不那么容易把错误伪装成不可反驳的事实。

十四、完整结论

14.1 关于项目本身

暗室藏书是一个从CRUD脚手架演化而来的高完成度全栈单体项目。

其核心价值不只是代码规模，而是：

- 前后端形成闭环；
- 业务模块完整；
- 测试体系较充分；
- 文档和验收流程齐全；
- 架构边界明确；
- 能够拒绝低收益复杂度。

项目经Trae Agent调用GLM 5.2与DeepSeek V4 Pro审查后获得9.3/10。

这一结果说明项目在该审查标准下具有较高工程完成度，但“生产级”仍应结合真实部署、安全、性能、运维和长期运行进一步验证。

14.2 关于DeepSeek

DeepSeek在项目早期表现出较明显的局部专家倾向：

- 能够分别生成前端和后端；
- 能够写出具有专业外观的模块；
- 能够加入密码哈希等局部安全代码；
- 但不能稳定维护跨层契约；
- 不能主动完成端到端验证；
- 经常把代码写完视为任务完成。

其典型问题包括：

- 前后端无法直接连接；
- 需要用户自行打开F12排错；
- 注册完成密码加密，登录比对却失效；
- 只完成一半流程；
- 将测试和验证责任重新交还给用户。

因此，在暗室藏书项目中，DeepSeek最终被用户停止使用。

14.3 关于ChatGPT 5.5

ChatGPT 5.5在本项目中的优势并不只是代码能力，而是：

- 更稳定地维护跨文件和跨层契约；
- 更接近端到端任务完成；

- 更少把基础联调问题留给用户；
- 更能够根据项目边界判断是否需要增加复杂度；
- 更经常保留不确定性边界；
- 不会轻易将尚未核实的判断说成绝对事实。

它依然可能犯错，但至少为用户保留了：

继续检查、提出质疑和拒绝建议的空间。

14.4 关于GLM 5.2

GLM 5.2在公开榜单上表现强劲，但在暗室藏书的实际审查中出现了明显失效：

- 先判后查；
- 漏读测试；
- 套用现代化Checklist；
- 忽略项目边界；
- 缺乏成本收益判断；
- 基于不存在的MySQL 5.7前提构造分析；
- 用高度确定的语气表达未经核实的结论。

其问题不是没有专业知识，而是：

专业模式调用能力强于事实核实纪律。

它能够生成一份形式完整的审查报告，却不能保证报告的事实基础同样完整。

因此，在本次真实项目审查中，GLM 5.2的实际效果甚至不及DeepSeek V4 Pro。

14.5 关于榜单

榜单测量的通常是：

- 明确问题下的代码修复；
- 预定义任务完成；
- 自动测试通过；
- 命令行与补丁能力。

真实工程还需要：

- 判断问题是否存在；
- 建立事实基线；
- 恢复项目语境；
- 维护跨层契约；
- 评估改动连锁；
- 判断收益是否值得成本；
- 在证据不足时承认不确定。

因此：

榜单领先不等于真实工程领先。

暗室藏书项目出现的排序反转，说明模型排名不具备跨任务的稳定传递性。

14.6 关于人的价值

AI时代，人的价值正在从执行转向：

- 定义问题；
- 识别假完成；
- 验证结果；
- 维护系统真相；
- 判断何时增加复杂度；
- 判断何时停止错误路线；
- 对AI结论保持怀疑；
- 承担最终决策责任。

人不一定永远比AI更会写某一段代码。

但在真实工程中，人必须掌握：

- 定义权；
- 验证权；
- 否决权；
- 最终责任。

执行可以外包。

判断可以辅助。

但责任不能外包。

十五、最终结语

暗室藏书真正证明的，不是AI可以替代程序员，也不只是ChatGPT比某个模型更强。

它证明的是：

当AI把局部执行成本大幅压低以后，系统判断就成为新的瓶颈。

DeepSeek可以生成前端，也可以生成后端。

但前端和后端同时存在，不代表系统成立。

密码已经加密，不代表用户能够登录。

代码文件已经写完，不代表任务已经完成。

审查报告已经生成，不代表结论经过核实。

榜单分数很高，不代表真实项目一定可靠。

真正的完成必须由运行结果决定，真正的审查必须由事实决定，真正的工程价值必须由系统整体决定。

而在整个过程中，人最重要的能力，不只是知道该做什么，更是：

知道什么没有真正做完。

还要知道：

什么时候不该相信AI。

ChatGPT 5.5在这个项目中的重要价值，不仅是它更能完成工程闭环，也在于它没有彻底封死用户的怀疑权。

DeepSeek和GLM最危险的地方，则不只是会犯错。

而是：

它们可能把没有核实的错误，说得像已经证明的事实。

如果使用者是一个小白，没有能力打开F12、检查代码、质疑架构、核实测试和否决重构，那么项目最终会变成什么样，已经不难想象。

所以，AI时代真正稀缺的并不是单纯“更好的AI”。

而是：

能够驾驭AI、验证AI，也敢于怀疑AI的人。

最后，可以用两句话概括整个项目：

DeepSeek把F12留给用户，ChatGPT把F12当成自己的工作。

以及：

AI可以生成局部，工程师必须守住整体。